SECTION C

Question 06

```
import pandas as pd

# Load the dataset
df = pd.read_csv(r'/content/amazon.csv')

# Define term lists (BoW Dictionary)
positive_terms = ["good", "great", "excellent", "terrific", "fun", "recommend", "love", "awesome"]
negative_terms = ["bad", "poor", "boring", "slow", "waste", "horrible", "disappointed", "worst"]

def analyze_sentiment(text):
    text = str(text).lower()
    has_pos = any(word in text for word in positive_terms)
    has_neg = any(word in text for word in negative_terms)
    return has_pos, has_neg

# Apply analysis
results = df['reviewText'].apply(lambda x: pd.Series(analyze_sentiment(x)))
df[['is_pos', 'is_neg']] = results

# Calculate Percentages
pos_perc = (df['is_pos'].sum() / len(df)) * 100
neg_perc = (df['is_neg'].sum() / len(df)) * 100

print(f"--- Management Summary Report ---")
print(f"Total Reviews Analyzed: {len(df)}")
print(f"Reviews with Positive Terms: {pos_perc:.2f}%")
print(f"Reviews with Negative Terms: {neg_perc:.2f}%")
```

```
--- Management Summary Report ---
Total Reviews Analyzed: 20000
Reviews with Positive Terms: 53.65%
Reviews with Negative Terms: 8.62%
```

Question 07

```
import numpy as np

class Perceptron:
    def __init__(self, weights, bias):
        self.weights = np.array(weights)
        self.bias = bias

    def weighted_sum(self, inputs):
        # Calculation: (w1*x1 + w2*x2 ... + wn*xn) + bias
        return np.dot(inputs, self.weights) + self.bias

    def step_function(self, z):
        # Binary activation: 1 if positive/zero, 0 if negative
        return 1 if z >= 0 else 0

    def predict(self, inputs):
        z = self.weighted_sum(inputs)
        return self.step_function(z)

# Example: Gates (e.g., AND gate)
model = Perceptron(weights=[0.5, 0.5], bias=-0.7)
print(f"Prediction for input [1, 1]: {model.predict([1, 1])}") # Returns 1
```

```
Prediction for input [1, 1]: 1
```

Question 08

```
import nltk
from nltk import pos_tag, ne_chunk, word_tokenize

nltk.download('punkt', quiet=True)
nltk.download('averaged_perceptron_tagger_eng', quiet=True)
```

```
nltk.download('maxent_ne_chunker', quiet=True)
nltk.download('words', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('maxent_ne_chunker_tab', quiet=True)

def discover_entities(text):
    # 1. Tokenize and Tag parts of speech
    tokens = word_tokenize(text)
    tags = pos_tag(tokens)

    # 2. Extract Named Entities (NER)
    tree = ne_chunk(tags)

    entities = []
    for leaf in tree:
        if hasattr(leaf, 'label'):
            name = " ".join(child[0] for child in leaf)
            entities.append((name, leaf.label()))
    return entities

news_sample = "Elon Musk visited the Tesla factory in Berlin yesterday."
print(discover_entities(news_sample))
```

```
[('Elon', 'PERSON'), ('Musk', 'PERSON'), ('Tesla', 'GPE'), ('Berlin', 'GPE')]
```

Question 09

```
import numpy as np

def predict_class(input_vector):
    # 12-element target patterns for classes X and O
    target_X = np.array([1,0,1, 0,1,0, 1,0,1, 1,1,1]) # Sample X pattern
    target_O = np.array([1,1,1, 1,0,1, 1,0,1, 1,1,1]) # Sample O pattern

    # Compute similarity (Euclidean distance or dot product)
    # Here we use dot product to find the highest match
    match_X = np.dot(input_vector, target_X)
    match_O = np.dot(input_vector, target_O)

    if match_X > match_O:
        return "Class X", match_X
    else:
        return "Class O", match_O

# Simulation
test_image_vector = np.random.choice([0, 1], size=12)
label, score = predict_class(test_image_vector)
print(f"The input image is predicted to be: {label}")
```

```
The input image is predicted to be: Class O
```